1. Which keyword is used to initiate a conditional branch in Python?

(A) when
(B) if
(C) check
(D) condition

2. What token terminates the conditional expression line in an if statement in Python?

(A) ;
(B) :
(C) then
(D) ->

3. How are code blocks demarcated inside conditional and loop bodies in Python?

(A) Curly braces { }
(B) Indentation (whitespace)
(C) begin and end keywords
(D) Parentheses ( )

4. Which keyword represents 'else if' in Python multi-way branch selection?

(A) elseif
(B) elsif
(C) elif
(D) else if

5. What is the evaluated boolean value of the expression 15 % 4?

(A) 1
(B) 2
(C) 3
(D) 4

6. Which module in Python's standard library provides pseudo-random number generation functions?

(A) math
(B) random
(C) os
(D) sys

7. Which function in the random module returns a random integer N such that a <= N <= b (inclusive)?

(A) random.rand()
(B) random.randint(a, b)
(C) random.uniform(a, b)
(D) random.range(a, b)

8. What does random.random() return in Python?

(A) A random integer between 0 and 100
(B) A random float in the half-open interval [0.0, 1.0)
(C) A random boolean
(D) A random choice from a list

9. Which comparison operator checks if two numeric operands are not equal?

(A) <>
(B) !=
(C) !==
(D) not ==

10. Which logical operator returns True only when both relational conditions evaluate to True?

(A) or
(B) and
(C) not
(D) xor

11. Which logical operator inverts the boolean truth value of an expression?

(A) invert
(B) not
(C) !
(D) ~

12. What is the result of the boolean expression True or False in Python?

(A) True
(B) False
(C) None
(D) 1

13. What is the result of the boolean expression True and not False?

(A) True
(B) False
(C) None
(D) 0

14. Which loop structure repeatedly executes a block as long as its controlling test condition remains True?

(A) for loop
(B) while loop
(C) do-while loop
(D) repeat-until loop

15. Does standard Python have a native 'do-while' loop construct?

(A) Yes, using do { } while (cond)
(B) No, Python does not have a native do-while construct
(C) Yes, using repeat: while
(D) Yes, in Python 3.10+

16. Which statement immediately terminates the execution of the innermost enclosing loop?

(A) continue
(B) pass
(C) break
(D) exit

17. Which statement skips the remainder of the current iteration and jumps to the next cycle of the loop?

(A) break
(B) continue
(C) pass
(D) skip

18. Which statement is a syntactical null-operation (no-op) used as a placeholder in empty code blocks?

(A) continue
(B) pass
(C) break
(D) dummy

19. What is the sequence of integers generated by list(range(5)) in Python?

(A) [1, 2, 3, 4, 5]
(B) [0, 1, 2, 3, 4]
(C) [0, 1, 2, 3, 4, 5]
(D) [1, 2, 3, 4]

20. What does range(2, 10, 3) generate?

(A) [2, 5, 8]
(B) [2, 4, 6, 8]
(C) [2, 5, 8, 11]
(D) [3, 6, 9]

21. What is the default start value in range(stop) if not specified?

(A) 1
(B) 0
(C) -1
(D) None

22. What is the default step increment in range(start, stop) if omitted?

(A) 0
(B) 1
(C) 2
(D) -1

23. What happens if the condition in a while loop never becomes False and no break is encountered?

(A) Interpreter crashes immediately with StackOverflow
(B) An infinite loop occurs
(C) SyntaxError is raised
(D) PVM exits after 1000 cycles

24. What is the output of print(10 % 2 == 0)?

(A) True
(B) False
(C) 0
(D) None

25. Which built-in function returns pairs of index and element while iterating over a collection?

(A) zip()
(B) enumerate()
(C) range()
(D) iter()

26. Which built-in function pairs elements from multiple iterables element-wise in lockstep?

(A) zip()
(B) pair()
(C) combine()
(D) map()

27. When does the else block attached to a while or for loop execute in Python?

(A) Every iteration of the loop
(B) Only if the loop completes normally without encountering a break
(C) Only if a break is encountered
(D) Whenever an exception occurs

28. What is the ternary conditional expression syntax in Python?

(A) condition ? x : y
(B) x if condition else y
(C) if condition then x else y
(D) choose(condition, x, y)

29. Which function in the random module selects a single random element from a non-empty sequence?

(A) random.sample()
(B) random.choice()
(C) random.select()
(D) random.pick()

30. Which function in the random module shuffles a mutable sequence in-place?

(A) random.shuffle()
(B) random.permute()
(C) random.reorder()
(D) random.mix()

31. What is the output of len(range(1, 10, 2))?

(A) 4
(B) 5
(C) 9
(D) 10

32. What does list(range(5, 0, -1)) produce?

(A) [5, 4, 3, 2, 1]
(B) [5, 4, 3, 2, 1, 0]
(C) [4, 3, 2, 1]
(D) [0, 1, 2, 3, 4, 5]

33. Which of the following values is evaluated as False in a boolean conditional context?

(A) []
(B) {}
(C) 0
(D) All of the above

34. Which of the following values is evaluated as True in a boolean context?

(A) '0'
(B) [0]
(C) (-1)
(D) All of the above

35. What is the result of 10 > 5 and 5 < 2 in Python?

(A) True
(B) False
(C) None
(D) TypeError

36. What is the result of 10 > 5 or 5 < 2 in Python?

(A) True
(B) False
(C) None
(D) 1

37. What is the value of x after: x = 10; x += 5?

(A) 10
(B) 15
(C) 5
(D) 50

38. Which loop is optimal when the exact number of iterations is known in advance?

(A) while loop
(B) for loop with range()
(C) infinite while loop
(D) recursion only

39. Which loop structure is optimal when looping depends on an external condition or sentinel input?

(A) for loop with range()
(B) while loop
(C) nested for loop
(D) list comprehension

40. What does random.seed(s) accomplish?

(A) Generates a cryptographically secure key
(B) Initializes the PRNG with a fixed seed for reproducible pseudo-random sequences
(C) Seeds RAM with zeros
(D) Deletes existing random history

41. What is the output of the following loop?\nx = 0\nwhile x < 5:\n x += 2\nprint(x)

(A) 4
(B) 5
(C) 6
(D) Infinite loop

42. Consider: for i in range(1, 4): print(i, end=''). What is displayed?

(A) 1234
(B) 123
(C) 0123
(D) 1 2 3

43. What is the output of:\nfor i in range(3):\n if i == 1: break\nelse:\n print('End')

(A) End
(B) 1
(C) No output is printed
(D) 0 End

44. What is the output of:\nfor i in range(3):\n if i == 5: break\nelse:\n print('Finished')

(A) Finished
(B) 0 1 2
(C) No output
(D) SyntaxError

45. What is the value of c after:\nc = 0\nfor i in range(3):\n for j in range(2):\n c += 1

(A) 5
(B) 6
(C) 9
(D) 3

46. What does the following snippet print?\na = 10; b = 20\nval = a if a > b else b\nprint(val)

(A) 10
(B) 20
(C) True
(D) False

47. What is printed by:\ns = 0\nfor i in range(1, 10):\n if i % 3 == 0:\n s += i\nprint(s)

(A) 9
(B) 18
(C) 45
(D) 12

48. What is the output of:\nx = 5\nwhile x > 0:\n x -= 1\n if x == 2: continue\n print(x, end=' ')

(A) 4 3 2 1 0
(B) 4 3 1 0
(C) 5 4 3 1 0
(D) 4 3 1

49. What is the output of:\nfor i, char in enumerate('CSE'):\n if i == 1: print(char)

(A) C
(B) S
(C) E
(D) 1

50. What is the output of:\nnames = ['A', 'B']; scores = [90, 80]\nfor n, s in zip(names, scores):\n print(f'{n}:{s}', end=' ')

(A) 90 B:80
(B) A:80 B:90
(C) ['A', 90]
(D) Error

51. If zip() is given two lists of unequal lengths (e.g. len 3 and len 5), when does iteration stop?

(A) Stops when the shortest iterable is exhausted
(B) Fills missing elements with None
(C) Raises a ValueError
(D) Loops infinitely

52. Which module in the standard library provides zip_longest() to fill missing elements with a default value?

(A) collections
(B) itertools
(C) functools
(D) operator

53. What does the expression [x * 2 for x in range(4) if x % 2 == 0] produce?

(A) [0, 4]
(B) [0, 2, 4, 6]
(C) [2, 4]
(D) [4, 8]

54. What is the output of:\ni = 1\nwhile i < 10:\n i *= 2\nprint(i)

(A) 8
(B) 16
(C) 10
(D) 32

55. What is the output of:\ncount = 0\nfor i in range(1, 5):\n for j in range(i):\n count += 1\nprint(count)

(A) 10
(B) 16
(C) 14
(D) 20

56. What is the result of De Morgan's Law equivalence: not (A and B)?

(A) (not A) and (not B)
(B) (not A) or (not B)
(C) not A and B
(D) A or B

57. What is the result of De Morgan's Law equivalence: not (A or B)?

(A) (not A) and (not B)
(B) (not A) or (not B)
(C) A and not B
(D) not A or B

58. What does random.sample([10, 20, 30, 40], 2) return?

(A) A list of 2 unique random elements chosen without replacement
(B) A list of 2 random elements chosen with replacement
(C) A 2D array
(D) The sum of 2 random elements

59. What does random.choices([1, 2, 3], k=2) perform differently from random.sample()?

(A) Choices selects with replacement; sample selects without replacement
(B) Sample selects with replacement; choices selects without
(C) Choices works only with integers
(D) Sample works only with strings

60. What is the output of:\nfor x in []:\n print('Loop')\nelse:\n print('Empty')

(A) Loop
(B) Empty
(C) Loop Empty
(D) No output

61. What is the output of:\nx = 0\nif x:\n print('Yes')\nelse:\n print('No')

(A) Yes
(B) No
(C) SyntaxError
(D) None

62. What is the output of:\nx = 'Python'\nif x:\n print('Truthy')\nelse:\n print('Falsy')

(A) Truthy
(B) Falsy
(C) None
(D) Error

63. What is the output of:\na = [1, 2, 3]\nwhile a:\n a.pop()\nprint(len(a))

(A) 3
(B) 0
(C) 1
(D) Infinite loop

64. What is the output of:\nfor i in range(3):\n pass\nprint(i)

(A) 2
(B) 3
(C) None
(D) NameError: i is not defined

65. What is the output of:\ntotal = 0\nfor i in range(5):\n if i == 3: break\n total += i\nprint(total)

(A) 3
(B) 6
(C) 10
(D) 0

66. What is the output of:\ntotal = 0\nfor i in range(5):\n if i == 3: continue\n total += i\nprint(total)

(A) 10
(B) 7
(C) 6
(D) 4

67. What is the output of print(list(range(0)))?

(A) []
(B) [0]
(C) [-1]
(D) ValueError

68. What does the expression 0 < x < 10 mean in Python?

(A) Equivalent to (0 < x) and (x < 10) with x evaluated once
(B) Equivalent to (0 < x) < 10 (comparing boolean with 10)
(C) SyntaxError
(D) Chained bitwise operation

69. In C or Java, 0 < x < 10 evaluates (0 < x) as 0 or 1, then compares with 10. How does Python differ?

(A) Python chains relational comparisons using short-circuit logical AND
(B) Python evaluates from right to left
(C) Python converts both to floating point
(D) Python raises a TypeError

70. What is the output of:\nx = 10\nprint('High' if x > 15 else 'Medium' if x > 5 else 'Low')

(A) 'High'
(B) 'Medium'
(C) 'Low'
(D) SyntaxError

71. What is the output of:\nfor i in range(1, 4):\n print(i * '*')

(A) *\n**\n***
(B) ***\n**\n*
(C) ****
(D) *

72. What is the output of:\nx = 1\nwhile x < 4:\n print(x, end='')\n x += 1

(A) 123
(B) 1234
(C) 0123
(D) 1 2 3

73. What does the modulus operator x % n accomplish in circular array indexing?

(A) Constrains index x strictly within the boundary [0, n - 1]
(B) Expands the array dynamically
(C) Checks if x is prime
(D) Inverts the array order

74. What is the output of: (13 % 12) in 12-hour clock modular arithmetic?

(A) 1
(B) 12
(C) 0
(D) 13

75. What is the output of: (-1 % 12) in Python modular arithmetic?

(A) 11
(B) -1
(C) 1
(D) 0

76. What algorithm does Python's standard random module use as its core PRNG engine?

(A) Linear Congruential Generator (LCG)
(B) Mersenne Twister (MT19937)
(C) Xorshift128+
(D) ChaCha20

77. What is the period of the Mersenne Twister PRNG implemented in Python's random module?

(A) 2^32 - 1
(B) 2^64 - 1
(C) 2^19937 - 1
(D) 2^128

78. Is the standard random module cryptographically secure for passwords or security tokens?

(A) Yes, it is immune to cryptanalysis
(B) No; use the secrets module for cryptographic randomness
(C) Yes, if seeded with time.time()
(D) Yes, in Python 3.11+

79. Which module introduced in Python 3.6 should be used for generating cryptographically secure tokens and secrets?

(A) crypto
(B) secrets
(C) ssl
(D) hashlib

80. What is the time complexity of range(start, stop, step) object creation in Python 3?

(A) O(N) where N is number of elements
(B) O(1) constant time
(C) O(log N)
(D) O(step)

81. What is the space complexity of a range object in Python 3?

(A) O(N) memory allocation
(B) O(1) constant memory storing only start, stop, step attributes
(C) O(log N)
(D) O(step^2)

82. What is the time complexity of the membership test: (999_999 in range(1_000_000)) in Python 3?

(A) O(N) linear scan
(B) O(1) arithmetic calculation
(C) O(log N) binary search
(D) O(sqrt(N))

83. How does Python 3 optimize (x in range(start, stop, step)) to achieve O(1) time complexity?

(A) By performing a mathematical bounds check and modulus step alignment test
(B) By hashing the range into a set
(C) By using a binary search tree
(D) By compiling the range into C array

84. What occurs when modifying a list while iterating directly over it: for x in my_list: my_list.remove(x)?

(A) Safe removal of all items
(B) Index shifting skips elements because internal loop pointer advances monotonically
(C) RuntimeError: list changed size during iteration
(D) Memory leak

85. What is the idiomatic way to safely remove elements from a list while iterating?

(A) Iterate over a shallow slice copy: for x in my_list[:]: or use list comprehension
(B) Use a while loop without index
(C) Delete items with del x
(D) Set items to None

86. In nested loops with depth D and inner work O(1), where each loop runs N times, what is the time complexity?

(A) O(N * D)
(B) O(N^D)
(C) O(D^N)
(D) O(N log D)

87. How can a program break completely out of multiple deeply nested loops in Python without using goto?

(A) Use a boolean flag, encapsulate loops inside a function and return, or raise a custom exception
(B) Use 'break 2' or 'break all'
(C) Use the 'continue all' keyword
(D) Python does not permit breaking nested loops

88. Does Python support multi-level break statements such as 'break 2' like PHP or Java labeled breaks?

(A) Yes, using break 2
(B) No, Python has no labeled breaks or multi-level break syntax
(C) Yes, using break(2)
(D) Yes, using exit(2)

89. What bytecode opcode does CPython emit for standard for-loops over iterables?

(A) FOR_ITER
(B) LOOP_START
(C) ITER_NEXT
(D) JUMP_LOOP

90. What protocol must an object implement to be iterable in a Python for-loop?

(A) The Iterable Protocol: __iter__() returning an iterator with __next__()
(B) The Indexable Protocol: __index__()
(C) The Sequence Protocol: __len__() only
(D) The Callable Protocol: __call__()

91. What exception signals the termination of iteration in Python's iterator protocol?

(A) IndexError
(B) StopIteration
(C) KeyError
(D) EOFError

92. What is the output of:\ndef gen():\n for i in range(2):\n yield i\ng = gen()\nprint(next(g), next(g))

(A) 0 1
(B) 1 2
(C) 0 0
(D) StopIteration

93. What occurs on a subsequent next(g) call when a generator has no more values to yield?

(A) Returns None
(B) Raises StopIteration exception
(C) Returns -1
(D) Restarts from the beginning

94. What is the result of using a float step in range(0, 5, 0.5)?

(A) Generates [0, 0.5, 1.0, ...]
(B) TypeError: 'float' object cannot be interpreted as an integer
(C) ValueError: float step
(D) Automatically rounds to 1

95. Which library function should be used for floating-point arithmetic ranges with non-integer steps?

(A) numpy.arange() or itertools.count()
(B) range.float()
(C) sys.frange()
(D) math.range()

96. What is the output of:\nx = 0\nfor i in range(100):\n for j in range(100):\n if i == j:\n x += 1\nprint(x)

(A) 100
(B) 10000
(C) 5050
(D) 0

97. What is the space complexity of an iterator created by itertools.islice()?

(A) O(N)
(B) O(1) lazy evaluation
(C) O(log N)
(D) O(N^2)

98. What is the output of the following chained comparison: print(1 < 2 < 3 > 2)?

(A) True
(B) False
(C) TypeError
(D) SyntaxError

99. What is the output of print(1 < 2 > 3)?

(A) False (since 1 < 2 is True, but 2 > 3 is False)
(B) True
(C) TypeError
(D) 1

100. What is the output of:\nmatrix = [[1, 2], [3, 4]]\nflat = [val for row in matrix for val in row]\nprint(flat)

(A) [1, 2, 3, 4]
(B) [[1, 2], [3, 4]]
(C) [1, 3, 2, 4]
(D) [4, 3, 2, 1]

SECTION B: DESCRIPTIVE & SUBJECTIVE QUESTIONS

Q1. Explain Python's Conditional Branching Architecture (if, if-else, if-elif-else). Discuss block scoping rules and short-circuit evaluation in complex boolean expressions.

Q2. Analyze Nested Conditionals versus Chained Comparisons and Flat Refactoring. Discuss the 'Dangling Else' problem and how Python's indentation eliminates it.

Q3. Explain the Ternary Conditional Operator (x if condition else y). Compare its syntax, readability, evaluation semantics, and nesting with traditional if-else blocks.

Q4. Explain the Iteration Protocol, While Loops, and Sentinel-Controlled Loops in Python. Detail condition evaluation, loop termination, and infinite loop prevention.

Q5. Explain the For Loop in Python and the Sequence Iteration Mechanism. Compare Python's for loop with traditional index-based for loops in C/Java.

Q6. Explain the range() type in Python 3. Detail its lazy evaluation, arithmetic progression parameters, and internal memory efficiency compared to Python 2 range.

Q7. Explain Break and Continue Statements. Detail how they alter control flow within while and for loops, and explain their interaction with the loop else clause.

Q8. Analyze Nested Loops and Algorithmic Complexity. Explain how nested iteration impacts time complexity (O(N^2), O(N^3)) and strategies to optimize nested loops.

Q9. Design an interactive Python program to simulate the Euclidean Algorithm for finding the Greatest Common Divisor (GCD) using while loops with execution trace.

Q10. Implement a Number Guessing Game in Python with binary search hints, random number generation, input validation, and attempt tracking.

Q11. Design a Python program to generate Pascal's Triangle up to N rows using nested loops and combinatorial binomial coefficients.

Q12. Implement a comprehensive Primality Testing program utilizing Trial Division up to sqrt(N) and the Sieve of Eratosthenes algorithm.

Q13. Implement a Python program for the Collatz Conjecture (3n + 1 Problem) with hailstone sequence generation, stopping time analysis, and peak value tracking.

Q14. Explain the built-in enumerate() and zip() functions. Contrast iterating with explicit index tracking versus idiomatic parallel iteration with code.

Q15. Design a Python program to print various geometric console patterns: Right-Angled Triangle, Centered Pyramid, Inverted Diamond, and Floyd's Triangle.

Q16. Implement a Banking ATM Transaction Simulator in Python using while loops, match-case (or if-elif), transaction logs, and PIN authentication.

Q17. Explain the random module in depth. Detail the differences between randint, randrange, uniform, choice, choices, sample, and shuffle.

Q18. Explain the secrets module introduced in Python 3.6. Why is random unsuitable for security, and how does secrets generate cryptographically secure tokens?

Q19. Implement a Binary Search algorithm using while loops. Analyze search intervals, mid calculation, logarithmic convergence, and asymptotic complexity.

Q20. Implement the Bubble Sort algorithm in Python using nested loops. Implement an early-exit optimization flag and analyze worst, average, and best-case complexities.

Q21. Design an interactive Caesar Cipher encryption and decryption program in Python using loops, character ordinals (ord), and modulo 26 arithmetic.

Q22. Implement a Matrix Transpose and Matrix Addition program in Python using nested loops and 2D list comprehensions.

Q23. Design a Python program to compute the Taylor Series expansion of sin(x) and cos(x) up to N terms using while loops and factorial accumulation.

Q24. Implement a Palindrome and Armstrong Number Validator in Python using numeric while loops without converting numbers to strings.

Q25. Implement an Run-Length Encoding (RLE) lossless string data compression utility using iterative while/for loops.

Q26. Explain the itertools module in Python. Demonstrate infinite iterators (count, cycle, repeat) and combinatoric generators (product, permutations, combinations).

Q27. Explain the Structural Pattern Matching (match-case) introduced in Python 3.10 (PEP 634). Compare with C-style switch-case and traditional if-elif-else.

Q28. Design a Python program to solve the 0/1 Knapsack Problem or Subset Sum Problem using iterative dynamic programming tables and nested loops.

Q29. Implement a Monte Carlo Simulation in Python to estimate the mathematical constant Pi (pi) using random numbers and circular geometry.

Q30. Implement Conway's Game of Life cellular automaton in Python using 2D grids, nested loops, and toroidal boundary conditions.

SECTION C: ANSWER KEY - MULTIPLE CHOICE QUESTIONS

Q1:	[B]	**Q21:**	[B]	**Q41:**	[C]	**Q61:**	[B]	**Q81:**	[B]
Q2:	[B]	**Q22:**	[B]	**Q42:**	[B]	**Q62:**	[A]	**Q82:**	[B]
Q3:	[B]	**Q23:**	[B]	**Q43:**	[C]	**Q63:**	[B]	**Q83:**	[A]
Q4:	[C]	**Q24:**	[A]	**Q44:**	[A]	**Q64:**	[A]	**Q84:**	[B]
Q5:	[C]	**Q25:**	[B]	**Q45:**	[B]	**Q65:**	[A]	**Q85:**	[A]
Q6:	[B]	**Q26:**	[A]	**Q46:**	[B]	**Q66:**	[B]	**Q86:**	[B]
Q7:	[B]	**Q27:**	[B]	**Q47:**	[B]	**Q67:**	[A]	**Q87:**	[A]
Q8:	[B]	**Q28:**	[B]	**Q48:**	[B]	**Q68:**	[A]	**Q88:**	[B]
Q9:	[B]	**Q29:**	[B]	**Q49:**	[B]	**Q69:**	[A]	**Q89:**	[A]
Q10:	[B]	**Q30:**	[A]	**Q50:**	[A]	**Q70:**	[B]	**Q90:**	[A]
Q11:	[B]	**Q31:**	[B]	**Q51:**	[A]	**Q71:**	[A]	**Q91:**	[B]
Q12:	[A]	**Q32:**	[A]	**Q52:**	[B]	**Q72:**	[A]	**Q92:**	[A]
Q13:	[A]	**Q33:**	[D]	**Q53:**	[A]	**Q73:**	[A]	**Q93:**	[B]
Q14:	[B]	**Q34:**	[D]	**Q54:**	[B]	**Q74:**	[A]	**Q94:**	[B]
Q15:	[B]	**Q35:**	[B]	**Q55:**	[A]	**Q75:**	[A]	**Q95:**	[A]
Q16:	[C]	**Q36:**	[A]	**Q56:**	[B]	**Q76:**	[B]	**Q96:**	[A]
Q17:	[B]	**Q37:**	[B]	**Q57:**	[A]	**Q77:**	[C]	**Q97:**	[B]
Q18:	[B]	**Q38:**	[B]	**Q58:**	[A]	**Q78:**	[B]	**Q98:**	[A]
Q19:	[B]	**Q39:**	[B]	**Q59:**	[A]	**Q79:**	[B]	**Q99:**	[A]
Q20:	[A]	**Q40:**	[B]	**Q60:**	[B]	**Q80:**	[B]	**Q100:**	[A]

MCQ Scoring Guide & Difficulty Distribution:

Questions 1 to 40: Foundational & Conceptual [Easy] | Questions 41 to 75: Analytical & Applied [Moderate]

Questions 76 to 100: Complex Numerical & Advanced Problem Solving [Difficult]

SECTION B: DETAILED SOLUTIONS FOR SUBJECTIVE QUESTIONS Explain the structure (if, if-else, if-elif-else). Discuss block scoping rules and short-circuit evaluation in co

```
  "Bracket": bracket_name,
  "Rate": f"{tax_rate * 100:.1f}%",
  "TaxPayable": calculated_tax
  }

  # Execution and verification
  print(evaluate_tax_bracket(750_000))
  print(evaluate_tax_bracket(1_500_000))
```

5. Memory Model & Complexity Analysis:
- Namespace: annual_income, tax_rate, bracket_name, and calculated_tax all reside within the local stack frame.
- Time Complexity: O(1) constant time bound by maximum 4 scalar branch comparisons.
- Space Complexity: O(1) auxiliary memory overhead.

Solution to Question 2: >>> Evaluate, Analyze Nested Conditionals versus Chained Comparisons and Flat Refactoring. Discuss the 'Dangling Else' problem and how Python's

```
  else:
  return "Missing User"

  # Clean Pattern: Flat Early-Return Guard Clauses
  def verify_user_flat(user):
  if user is None:
  return "Missing User"
  if not user.get("is_authenticated"):
  return "Unauthenticated"
  if user.get("role") != "admin":
  return "Unauthorized Role"
  return "Admin Access Granted"

  # Test verification
  sample_admin = {"is_authenticated": True, "role": "admin"}
  print("Flat Guard Result:", verify_user_flat(sample_admin))
```

5. Complexity Analysis:
- Time Complexity: O(1) worst-case 3 dictionary lookups.
- Space Complexity: O(1) auxiliary frame memory.

>>P Solution to Question 3: Explain the Ternary Conditional Operator (x if condition else y). Compare its syntax, readability, evaluation semantics, and nesting with

```
# 3. Guarding against ZeroDivisionError inside an expression
num = 100
denom = 0
safe_ratio = (num / denom) if denom != 0 else float('inf')
print(f"Safe Ratio (denom=0): {safe_ratio}")
```

4. Complexity Analysis:
- Time Complexity: O(1) branch evaluation.
- Space Complexity: O(1) storage for evaluated return value.

Solution to Question 4:

>>> Explain the Iteration Protocol, While Loops, and Sentinel-Controlled Loops in Python. Detail condition evaluation, loop termination, and

```
    avg = sum(valid_readings) / len(valid_readings) if valid_readings else 0.0
    return {"Count": len(valid_readings), "Average": round(avg, 2), "Data": valid_readings}

    sensor_stream = [24.5, 25.1, 24.8, 26.0, -999, 30.5, 40.2]
    report = simulate_sensor_monitoring(sensor_stream)
    print("Sensor Processing Report:", report)
```

5. Complexity Analysis:
- Time Complexity: O(K) where K <= N is the index of the sentinel token.
- Space Complexity: O(K) memory to store valid readings.

Solution to Question 5:

>>> Explain the For Loop in Python and the Sequence Iteration Mechanism. Compare Python's for loop with traditional index-based for loop

```
try:
item = next(iterator)
print(f" Manual next(): {item}")
except StopIteration:
break # Clean termination
```

4. Complexity Analysis:
- Time Complexity: O(N) where N is the number of elements in the collection.
- Space Complexity: O(1) auxiliary memory for the iterator object.

Solution to Question 6:

>>> **Solution to Question 6:** Explain the range() type in Python 3. Detail its lazy evaluation, arithmetic progression parameters, and internal memory efficiency comp

```
# O(1) Membership testing
print(f"Is 2_500_000 in r_huge? {2_500_000 in r_huge}")

5. Complexity Analysis:
- Time Complexity: O(1) creation, indexing, membership check, and slicing.
- Space Complexity: O(1) strictly constant memory.
```

>> Solution to Question 57: Explain Break and Continue Statements. Detail how they alter control flow within while and for loops, and explain their interaction with t

4. Complexity Analysis:
- Primality Test Time Complexity: O(sqrt(N)) best/worst case.
- Space Complexity: O(1) auxiliary variable storage.

Solution to Question 8:

>>> **Solution to Question 8:** Analyze Nested Loops and Algorithmic Complexity. Explain how nested iteration impacts time complexity (O(N^2), O(N^3)) and strateg

```
print("Matrix Multiplication Result:")
for row in multiply_matrices(A, B):
print(" ", row)
```

5. Complexity Analysis:
- Time Complexity: O(N^3) for N x N matrices.
- Space Complexity: O(N^2) memory for the output product matrix.

>> Solution to Question 9: Design an interactive Python program to simulate the Euclidean Algorithm for finding the Greatest Common Divisor (GCD) using while l

- Time Complexity: O(log(min(a, b))) by Lame's Theorem (number of divisions <= 5 * number of decimal digits).
- Space Complexity: O(1) scalar integer registers.

Solution to Question 10:

```
>>> # Implement a Number Guessing Game in Python with binary search hints, random number generation, input validation, and attempt tracking.
```

```
    else:
    print(f" BINGO! Correct guess {user_guess} found in {attempts} attempts!")
    break

    return {"Target": secret_number, "TotalAttempts": attempts, "History": guess_history}

    # Simulated test run demonstrating binary search convergence
    test_secret = 68
    simulated_binary_search = [50, 75, 62, 68]
    play_number_guessing_game(secret_number=test_secret, simulated_guesses=simulated_binary_search)

    3. Complexity Analysis:
    - Optimal Strategy Time Complexity: O(log2(N)) = 7 attempts for N=100 via Binary Search.
    - Space Complexity: O(attempts) to store guess history.
```

>> Solution to Question 11: Design a Python program to generate Pascal's Triangle up to N rows using nested loops and combinatorial binomial coefficients.

```
    return triangle

    # Verification
    triangle_output = generate_pascals_triangle(6)
```

3. Memory Model & Complexity Analysis:
- Total Elements: 1 + 2 + 3 + ... + N = N*(N + 1)/2 entries.
- Time Complexity: O(N^2) additions.
- Space Complexity: O(N^2) auxiliary memory to store triangle rows.

Solution to Question 12:

>>> Implement a comprehensive Primality Testing program utilizing Trial Division up to sqrt(N) and the Sieve of Eratosthenes algorithm.

```
# Demonstration
test_num = 997
print(f"Trial Division: Is {test_num} Prime? -> {is_prime_trial(test_num)}")

primes_under_50 = sieve_of_eratosthenes(50)
print(f"Sieve: Primes up to 50: {primes_under_50}")
```

3. Complexity Analysis:
- Trial Division Time Complexity: O(sqrt(N)) single number test.
- Sieve Time Complexity: O(M log(log M)) for all primes up to M.
- Sieve Space Complexity: O(M) boolean flags array.

Solution to Question 13:

>>> Implement a Python program for the Collatz Conjecture (3n + 1 Problem) with hailstone sequence generation, stopping time analysis, a

3. Complexity Analysis:
- Time Complexity: Unproven rigorously; empirically O(Steps) where steps vary widely.
- Space Complexity: O(Steps) to store trajectory points in memory.

---

2. Controls character printing (asterisks, numbers, characters).

2. Python Implementation:

```
class PatternGenerator:
@staticmethod
def right_triangle(rows: int):
print(f"--- Right Triangle ({rows} rows) ---")
for i in range(1, rows + 1):
print("*" * i)

@staticmethod
def centered_pyramid(rows: int):
print(f"\n--- Centered Pyramid ({rows} rows) ---")
for i in range(1, rows + 1):
spaces = " " * (rows - i)
stars = "*" * (2 * i - 1)
print(spaces + stars)

@staticmethod
def inverted_diamond(rows: int):
print(f"\n--- Diamond Pattern ({rows} rows) ---")
# Top half
for i in range(1, rows + 1):
print(" " * (rows - i) + "*" * (2 * i - 1))
# Bottom half
for i in range(rows - 1, 0, -1):
print(" " * (rows - i) + "*" * (2 * i - 1))

@staticmethod
def floyds_triangle(rows: int):
print(f"\n--- Floyd's Triangle ({rows} rows) ---")
num = 1
for i in range(1, rows + 1):
for _ in range(i):
print(f"{num:2}", end=" ")
num += 1
print()

# Verification
PatternGenerator.right_triangle(4)
PatternGenerator.centered_pyramid(4)
PatternGenerator.floyds_triangle(4)
```

3. Complexity Analysis:
- Time Complexity: O(rows^2) characters formatted and printed.
- Space Complexity: O(rows) string buffering per line.

>>Solution to Question 16:

Implement a Banking ATM Transaction Simulator in Python using while loops, match-case (or if-elif), transaction logs, and PIN authenti

provides a multi-option transaction menu (Check Balance, Deposit, Withdraw, Mini-Statement, Exit), maintains an audit trail log, and guards against negative inputs or overdrafts.

2. Python Implementation:

```
class ATMSimulator:
def __init__(self, initial_pin="1234", initial_balance=5000.0):
self._pin = initial_pin
self._balance = initial_balance
self._transaction_log = [f"Account opened with initial balance: ${initial_balance:.2f}"]

def authenticate(self, entered_pin: str) -> bool:
return entered_pin == self._pin

def run_simulation(self, commands):
print("=" * 55)
print(" WELCOME TO PYTHON SECURE BANK ATM ")
print("=" * 55)

cmd_iter = iter(commands)

while True:
try:
action = next(cmd_iter)
except StopIteration:
break

op = action.get("type")

if op == "BALANCE":
print(f"[ACTION] Current Available Balance: ${self._balance:,.2f}")
elif op == "DEPOSIT":
amount = action.get("amount", 0.0)
if amount <= 0:
print("[ERROR] Deposit amount must be positive.")
else:
self._balance += amount
self._transaction_log.append(f"Deposited: +${amount:.2f}")
print(f"[SUCCESS] Deposited ${amount:,.2f}. New Balance: ${self._balance:,.2f}")
elif op == "WITHDRAW":
amount = action.get("amount", 0.0)
if amount > self._balance:
print(f"[REJECTED] Overdraft limit exceeded. Balance: ${self._balance:,.2f}")
elif amount <= 0:
print("[ERROR] Withdrawal amount must be positive.")
else:
self._balance -= amount
self._transaction_log.append(f"Withdrawn: -${amount:.2f}")
print(f"[SUCCESS] Dispensed ${amount:,.2f}. Remaining Balance: ${self._balance:,.2f}")
elif op == "STATEMENT":
print("--- MINI STATEMENT AUDIT LOG ---")
for entry in self._transaction_log:
print(f" * {entry}")
elif op == "EXIT":
print("Session terminated. Thank you for banking with us!")
```

```
  break

  # Automated Test Run
  atm = ATMSimulator()
  test_script = [
  {"type": "BALANCE"},
  {"type": "DEPOSIT", "amount": 2500.0},
  {"type": "WITHDRAW", "amount": 1200.0},
  {"type": "WITHDRAW", "amount": 10000.0}, # Insufficient funds
  {"type": "STATEMENT"},
  {"type": "EXIT"}
  ]
  atm.run_simulation(test_script)
```

3. Complexity Analysis:
- Time Complexity: O(1) per transaction request.
- Space Complexity: O(T) where T is total transaction history entries.

>> Solution to Question 17: Explain the random module in depth. Detail the differences between randint, randrange, uniform, choice, choices, sample, and shuffle.

```
random.shuffle(deck)
print(f"shuffle(in-place): {deck}")

4. Complexity Analysis:
- Time Complexity: O(1) for randint/choice; O(k) for choices/sample; O(N) for shuffle.
- Space Complexity: O(1) for in-place shuffle; O(k) for sample/choices.
```

Solution to Question 18:

```
>>> # Explain the secrets module introduced in Python 3.6. Why is random unsuitable for security, and how does secrets generate cryptograp
```

- Time Complexity: O(length) entropy extraction.
- Space Complexity: O(length) string memory.

>>> Solution to Question 19: Implement a Binary Search algorithm using while loops. Analyze search intervals, mid calculation, logarithmic convergence, and asymp

```
print(f"Index of {target_val}: {result_idx}")
```

3. Complexity Analysis:
- Best-Case Time: O(1) (target at exact middle on first step).
- Worst-Case Time: O(log2 N) (search space halves every iteration).
- Auxiliary Space: O(1) strictly constant space.

>> Solution to Question 20: Implement the Bubble Sort algorithm in Python using nested loops. Implement an early-exit optimization flag and analyze worst, averag

```
# Best-Case Test (Already Sorted)
already_sorted = [1, 2, 3, 4, 5]
_, best_comps, best_swaps = bubble_sort_optimized(already_sorted)
print(f"Already Sorted Test -> Comps: {best_comps}, Swaps: {best_swaps} (O(N) Best Case!)")
```

4. Complexity Analysis:
- Best-Case Time: O(N) (1 pass of N - 1 comparisons).
- Average & Worst-Case Time: O(N^2) comparisons and swaps.
- Auxiliary Space: O(1) in-place sorting.

>> Solution to Question 21: Design an interactive Caesar Cipher encryption and decryption program in Python using loops, character ordinals (ord), and modulo 26

```
recovered = CaesarCipher.decrypt(cipher, shift_key)

print(f"Original Text: '{plaintext}'")
print(f"Encrypted (k={shift_key}): '{cipher}'")
print(f"Decrypted Text: '{recovered}'")

3. Complexity Analysis:
- Time Complexity: O(N) single pass over text of length N.
- Space Complexity: O(N) character buffer.
```

Question 22: Implement a Matrix Transpose and Matrix Addition program in Python using nested loops and 2D list comprehensions.

Solution:

```
print("Transpose of Matrix 1:")
for r in MatrixOperations.transpose_comprehension(mat1): print(" ", r)

print("Matrix 1 + Matrix 2:")
for r in MatrixOperations.add_matrices(mat1, mat2): print(" ", r)
```

3. Complexity Analysis:
- Time Complexity: O(R * C) where R is rows and C is columns.
- Space Complexity: O(R * C) memory allocated for result matrix.

>> Solution to Question 23: Write a Python program to compute the Taylor Series expansion of sin(x) and cos(x) up to N terms using while loops and factorial acc

---

Solution to Question 24:

```
>>> # Palindrome Number Validator
>>> num = int(input("Enter a number: "))
Enter a number: 121
>>> original = num
>>> reversed_num = 0
>>> while num > 0:
...     digit = num % 10
...     reversed_num = reversed_num * 10 + digit
...     num = num // 10
...
>>> if original == reversed_num:
...     print("Palindrome")
... else:
...     print("Not a Palindrome")
...
Palindrome
>>> # Armstrong Number Validator
>>> num = int(input("Enter a number: "))
Enter a number: 153
>>> original = num
>>> sum = 0
>>> while num > 0:
...     digit = num % 10
...     sum = sum + digit ** 3
...     num = num // 10
...
>>> if original == sum:
...     print("Armstrong Number")
... else:
...     print("Not an Armstrong Number")
...
Armstrong Number
```

```
{NumberValidator.is_armstrong(9474)}")
```

3. Complexity Analysis:
- Time Complexity: O(D) where D = floor(log10 N) + 1 is the number of decimal digits.
- Space Complexity: O(1) arithmetic integer registers.

Question 25: Implement an Run-Length Encoding (RLE) lossless string data compression utility using iterative while/for loops.

Solution:

```
print(f"Compression Ratio: {len(compressed_data) / len(sample_data) * 100:.1f}%")
print(f"Lossless Recovery: {decompressed_data == sample_data}")
```

3. Complexity Analysis:
- Time Complexity: O(N) single linear pass over the input string.
- Space Complexity: O(N) memory for the output compressed string.

>> Solution to Question 28: Explain the itertools module in Python. Demonstrate infinite iterators (count, cycle, repeat) and combinatoric generators (product, permu

[Problem Statement]: Explain the Structural Pattern Matching (match-case) introduced in Python 3.10 (PEP 634). Compare with C-style switch-ca
[Detailed Step-by-Step Answer]:
1. Paradigm Shift: PEP 634 Structural Pattern Matching:
Introduced in Python 3.10, match-case is fundamentally more powerful than C-style switch-case statements:
- C switch: Only compares a primitive integer/char against static compile-time constants. Requires explicit break statements to avoid fall-through.
- Python match-case: Inspects both value and structure (shapes of sequences, objects, dictionaries), extracts variables, and matches against patterns with optional guard clauses (if). No break is required; execution never falls through.

2. Pattern Types Supported:
- Literal patterns: case 404:
- Capture patterns: case (x, y):
- Wildcard pattern: case _: (Matches anything; default branch)
- OR patterns: case 401 | 403:
- Guard clauses: case x if x > 0:

3. Python Code Demonstration:

```
def handle_rest_api_event(event: dict) -> str:
"""Process API requests using structural pattern matching."""
match event:
case {"action": "login", "user": username, "status": "success"}:
return f"User '{username}' logged in successfully."
case {"action": "login", "user": username, "status": "failed", "attempts": n} if n >= 3:
return f"ALERT: Account '{username}' locked due to {n} failed attempts!"
case {"action": "fetch", "resource_id": int(rid)}:
return f"Fetching resource record #{rid}."
case {"action": "logout"}:
return "User logged out."
case _:
return "Unknown event payload received."

# Demonstration
print(handle_rest_api_event({"action": "login", "user": "Hariom", "status": "success"}))
print(handle_rest_api_event({"action": "login", "user": "Admin", "status": "failed",
"attempts": 4}))
print(handle_rest_api_event({"action": "fetch", "resource_id": 1042}))
```

4. Complexity Analysis:
- Time Complexity: O(P) pattern checks matching structure.
- Space Complexity: O(1) auxiliary frame memory.

>> Solution to Question 28: [Problem Statement]: Design a Python program to solve the 0/1 Knapsack Problem or Subset Sum Problem using iterative dynamic programming tables and

- If w[i - 1] > w: DP[i][w] = DP[i - 1][w]
- Else: DP[i][w] = max(DP[i - 1][w], v[i - 1] + DP[i - 1][w - w[i - 1]])

2. Python Implementation:

```
def solve_01_knapsack(values: list, weights: list, capacity: int) -> tuple:
n = len(values)
# Initialize DP table (N + 1) x (capacity + 1) with zeros
dp = [[0 for _ in range(capacity + 1)] for _ in range(n + 1)]

# Bottom-up iterative table construction
for i in range(1, n + 1):
item_val = values[i - 1]
item_wt = weights[i - 1]
for w in range(1, capacity + 1):
if item_wt <= w:
dp[i][w] = max(dp[i - 1][w], item_val + dp[i - 1][w - item_wt])
else:
dp[i][w] = dp[i - 1][w]

max_val = dp[n][capacity]

# Backtrack to find selected items
selected_items = []
curr_w = capacity
for i in range(n, 0, -1):
if dp[i][curr_w] != dp[i - 1][curr_w]:
selected_items.append(i - 1) # 0-indexed item
curr_w -= weights[i - 1]

selected_items.reverse()
return max_val, selected_items

# Verification
vals = [60, 100, 120]
wts = [10, 20, 30]
cap = 50

best_val, chosen = solve_01_knapsack(vals, wts, cap)
print(f"Knapsack Capacity: {cap} kg")
print(f"Maximum Profit Achieved: ${best_val}")
print(f"Selected Item Indices: {chosen} (Weights: {[wts[i] for i in chosen]})")
```

3. Complexity Analysis:
- Time Complexity: O(N * W) where N is number of items and W is capacity.
- Space Complexity: O(N * W) memory for the 2D DP matrix.

>> Solution to Question 29: Implement a Monte Carlo Simulation in Python to estimate the mathematical constant Pi (pi) using random numbers and circular geome

Ratio of Circle Area to Square Area:
Area_Circle / Area_Square = (pi * R^2) / (4 * R^2) = pi / 4.
Therefore:
pi = 4 * (Points_Inside_Circle / Total_Points).

2. Simulation Algorithm:
1. Generate N uniform random points (x, y) in the range [-1.0, 1.0].
2. For each point, evaluate Euclidean distance from origin: dist_sq = x^2 + y^2.
3. If dist_sq <= 1.0, the point falls inside the inscribed circle.
4. Estimate pi = 4.0 * inside_count / N.

3. Python Implementation:

```
import random
import math

def monte_carlo_pi_estimator(num_samples=500_000, seed=42):
random.seed(seed)
inside_circle = 0

for _ in range(num_samples):
x = random.uniform(-1.0, 1.0)
y = random.uniform(-1.0, 1.0)

if (x * x) + (y * y) <= 1.0:
inside_circle += 1

estimated_pi = 4.0 * inside_circle / num_samples
error = abs(estimated_pi - math.pi)

print("=" * 55)
print(" MONTE CARLO PI SIMULATION EXPERIMENT ")
print("=" * 55)
print(f"Total Random Samples (N): {num_samples:,}")
print(f"Points Inside Circle (M): {inside_circle:,}")
print(f"Estimated Pi Value: {estimated_pi:.6f}")
print(f"Actual math.pi Reference: {math.pi:.6f}")
print(f"Absolute Estimation Error: {error:.6f} ({error / math.pi * 100:.4f}%)")
print("=" * 55)
return estimated_pi

# Run verification
monte_carlo_pi_estimator(num_samples=250_000)
```

4. Complexity Analysis:
- Time Complexity: O(N) linear time proportional to sample count.
- Space Complexity: O(1) auxiliary memory registers.

>> Solution to Question 30: Implement Conway's Game of Life cellular automaton in Python using 2D grids, nested loops, and toroidal boundary conditions.

At each discrete tick, the fate of each cell is determined by its 8 orthogonal and diagonal neighbors:
1. Underpopulation: Any live cell with < 2 live neighbors dies.
2. Survival: Any live cell with 2 or 3 live neighbors lives to the next generation.
3. Overpopulation: Any live cell with > 3 live neighbors dies.
4. Reproduction: Any dead cell with exactly 3 live neighbors becomes a live cell.

2. Toroidal Boundary Conditions:
Edges wrap around using modulus arithmetic: (r + dr) % rows and (c + dc) % cols.

3. Python Implementation:

```
class GameOfLife:
def __init__(self, grid):
self.grid = grid
self.rows = len(grid)
self.cols = len(grid[0])

def count_neighbors(self, r: int, c: int) -> int:
count = 0
for dr in (-1, 0, 1):
for dc in (-1, 0, 1):
if dr == 0 and dc == 0:
continue
# Toroidal modular wraparound
nr = (r + dr) % self.rows
nc = (c + dc) % self.cols
count += self.grid[nr][nc]
return count

def step(self):
next_grid = [[0 for _ in range(self.cols)] for _ in range(self.rows)]
for r in range(self.rows):
for c in range(self.cols):
live_neighbors = self.count_neighbors(r, c)
if self.grid[r][c] == 1:
next_grid[r][c] = 1 if live_neighbors in (2, 3) else 0
else:
next_grid[r][c] = 1 if live_neighbors == 3 else 0
self.grid = next_grid

def display(self, label=""):
print(f"\n--- {label} ---")
for row in self.grid:
print(" ".join("#" if cell else "." for cell in row))

# Demonstration: Blinker Oscillator (Period 2)
blinker = [
[0, 0, 0, 0, 0],
[0, 0, 1, 0, 0],
[0, 0, 1, 0, 0],
[0, 0, 1, 0, 0],
[0, 0, 0, 0, 0]
]
```

```
sim = GameOfLife(blinker)
sim.display("Gen 0 (Vertical Blinker)")
sim.step()
sim.display("Gen 1 (Horizontal Blinker)")
sim.step()
sim.display("Gen 2 (Returned to Vertical)")
```

4. Complexity Analysis:
- Time Complexity: O(R * C) per time step evolution.
- Space Complexity: O(R * C) memory for next grid state.
